LAPORAN

Praktikum Artificial Intelligence

Klasifikasi Status Akademik Mahasiswa (Modul 1A & Modul 2A)

Oleh:

Ario veisa Rayanda Utomo (1204230086)

S1 SISTEM INFORMASI
FAKULTAS DIREKTORAT KAMPUS SURABAYA

TELKOM UNIVERSITY SURABAYA

2026

0. Praktik Modul 1A (Deep Neural Network)

Dataset dan Pembagian Data

Pada Modul 1A, model Deep Neural Network (DNN) dilatih untuk memprediksi status akademik mahasiswa (Aman, Waspada, Berisiko). Dataset yang digunakan adalah dataset_status_akademik.csv yang berisi 1.000 baris data dengan 6 fitur: ipk, kehadiran, jam_belajar, tugas_terkumpul, nilai_uts, dan aktif_organisasi. Data sudah dipisah lewat kolom bagian menjadi 800 data train dan 200 data test. Sebanyak 200 data test ini dipakai sebagai data uji yang sama pada evaluasi model DNN maupun model LLM di Modul 2A.

Pipeline dan Model DNN

Preprocessing dan model disatukan ke dalam satu Pipeline scikit-learn. Fitur numerik diskalakan dengan StandardScaler, sedangkan fitur aktif_organisasi diubah dengan OneHotEncoder. Model DNN menggunakan MLPClassifier dengan 3 hidden layer (64, 32, 16 neuron), aktivasi ReLU, solver Adam, regularisasi L2 alpha=0.001, dan early stopping (10% data train dipakai untuk validasi internal).

```
  preprocess = ColumnTransformer([
    ("num", StandardScaler(), ["ipk", "kehadiran", "jam_belajar", "tugas_terkumpul", "nilai_uts"]),
    ("cat", OneHotEncoder(handle_unknown="ignore"), ["aktif_organisasi"])
])
dnn = MLPClassifier(
    hidden_layer_sizes=(64, 32, 16), activation="relu", solver="adam", alpha=1e-3,
    learning_rate_init=1e-3, max_iter=500, early_stopping=True, validation_fraction=0.1, random_state=42
)
model = Pipeline([("preprocess", preprocess), ("dnn", dnn)])
```

Hasil Training dan Evaluasi Test

Proses training berjalan hingga epoch ke-50 dan berhenti karena early stopping dengan akurasi validasi terbaik 87.50%. Pada 200 data test, model DNN memperoleh akurasi 84.00% dan F1 macro 0.8269. Hasil ini melebihi target gerbang kelulusan modul (F1 >= 0.80) dan jauh di atas baseline tebakan terbanyak (F1 baseline = 0.1800). Model berhasil disimpan ke file model_status_akademik.pkl bersama ringkasan model_info.json.

Kelas	Precision	Recall	F1-Score	Support	Hasil
Aman	0.88	0.88	0.88	74	Bagus
Waspada	0.73	0.67	0.70	55	Paling sulit
Berisiko	0.88	0.93	0.90	71	Sangat bagus
Macro Avg	0.83	0.83	0.83	200	Lolos (>= 0.80)

Pengujian Model Mandiri (cek_model.py)

File model_status_akademik.pkl diuji dengan script cek_model.py pada satu contoh mahasiswa: IPK 3.45, Kehadiran 92%, Jam belajar 12, Tugas 95%, UTS 82, Aktif organisasi Ya. Model memprediksi status Aman dengan probabilitas: Aman = 0.995, Waspada = 0.005, Berisiko = 0.000. File model terbukti valid dan siap dideploy ke Streamlit pada Modul 1B.

1. Tabel Perbandingan DNN vs LLM dan Analisis Evaluasi

Pada Modul 2A, 200 data test yang sama diuji menggunakan AI model service (LLM). Konfigurasi LLM yang dipakai adalah prompt terbaik dari tahap validasi, yaitu Iterasi 2 (few-shot 6 contoh data train). Berikut tabel perbandingan hasil evaluasi antara DNN Modul 1A dan LLM Service 1 (ai-practicum):

Metrik Evaluasi	DNN (Modul 1A)	LLM S1 (Modul 2A)	Keterangan
Akurasi Keseluruhan	0.8400 (84.00%)	0.6100 (61.00%)	DNN lebih tinggi +23%
F1-Score (Macro)	0.8269 (82.69%)	0.6098 (60.98%)	DNN lebih tinggi +21.7%
Precision (Macro)	0.8280 (82.80%)	0.7760 (77.60%)	DNN lebih tinggi
Recall (Macro)	0.8269 (82.69%)	0.6337 (63.37%)	DNN lebih tinggi
F1 Aman	0.8784	0.7419	DNN lebih baik
F1 Waspada	0.6981	0.5667	DNN lebih baik
F1 Berisiko	0.9041	0.5208	DNN jauh lebih baik
Jawaban Invalid	0 (0.0%)	0 (0.0%)	Semua format rapi
Status Gerbang	Lolos (>= 0.80)	Tidak Lolos (< 0.70)	Hanya DNN lolos gerbang

Perbandingan Confusion Matrix

Hasil tebakan kedua model pada 200 data uji (74 Aman, 55 Waspada, 71 Berisiko) adalah sebagai berikut:

Asli	DNN: Aman	DNN: Wasp	DNN: Risk	LLM: Aman	LLM: Wasp	LLM: Risk
Aman (74)	65	9	0	46	28	0
Waspada (55)	9	37	9	4	51	0
Berisiko (71)	0	5	66	0	46	25

Analisis Hasil Evaluasi Data Test

1. Akurasi DNN Jauh Lebih Tinggi: DNN mencapai 84.00% sedangkan LLM hanya 61.00%. DNN memproses angka secara langsung dan menyesuaikan bobot matematisnya lewat data train 800 baris. Model DNN bisa memisahkan batas angka yang tipis (misalnya batas kehadiran 75% atau IPK 2.75). Sebaliknya, LLM membaca data sebagai teks biasa dan hanya mengandalkan pemahaman bahasa, sehingga kesulitan menentukan batas kuantitatif yang ketat.

2. LLM Terlalu Sering Menebak Waspada: Pada confusion matrix terlihat bahwa LLM Service 1 menebak Waspada sebanyak 125 kali dari 200 data (lebih dari 60% total tebakan). Dari 71 mahasiswa yang sebenarnya Berisiko, 46 di antaranya justru ditebak Waspada. Hal ini membuat recall kelas Berisiko anjlok ke 35.21%. Kesalahan ini berbahaya jika diterapkan di kampus karena mahasiswa yang terancam gagal studi tidak terdeteksi sejak awal.

3. Tidak Ada Kesalahan Ekstrem: Sisi baiknya, kedua model tidak pernah menukar Aman dan Berisiko secara langsung (0 mahasiswa Aman ditebak Berisiko, dan 0 mahasiswa Berisiko ditebak Aman). Namun DNN jauh lebih tegas: DNN mendeteksi 66 dari 71 mahasiswa Berisiko (recall 93%), sedangkan LLM hanya menemukan 25 mahasiswa.

2. Analisis Kelebihan dan Kekurangan DNN vs LLM

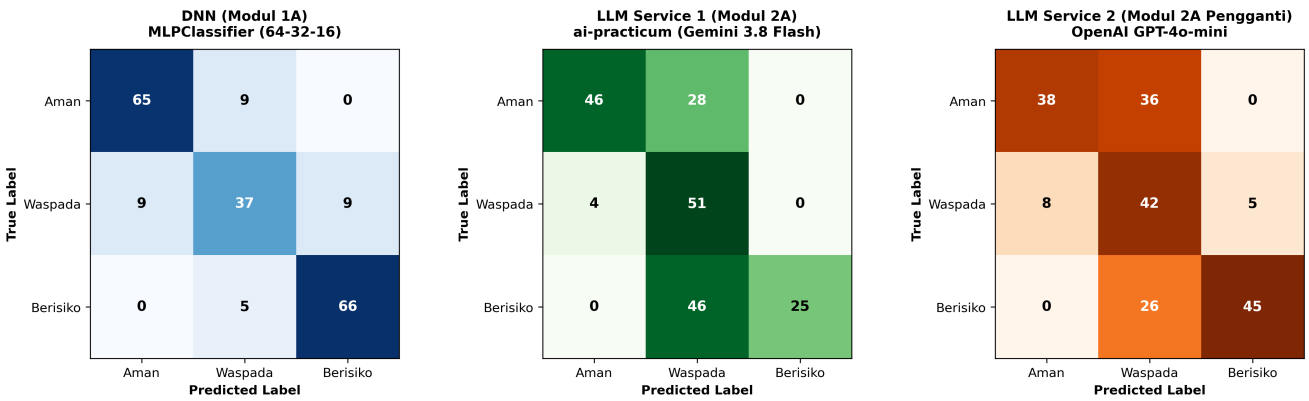
Berikut perbandingan kelebihan dan kekurangan antara DNN (Modul 1A) dan LLM Service (Modul 2A) berdasarkan hasil praktikum:

Aspek	DNN (Modul 1A)	LLM Service (Modul 2A)
Akurasi Data Tabular	Tinggi (84.00%). Mampu mempelajari pola hubungan angka non-linear dengan batas tegas.	Sedang (61.00%). Kurang peka membedakan angka-angka yang berada di dekat batas ambang.
Kelas Tersulit	Waspada (F1 0.6981) karena posisinya berada di zona peralihan antara Aman dan Berisiko.	Berisiko (Recall 35.21%) dan Waspada (Precision 40.80%) karena model terlalu sering memilih jawaban tengah.
Biaya (Cost)	Gratis. Model dijalankan di komputer lokal tanpa biaya request atau token.	Berbayar per request/token API. Jika dipakai untuk ribuan data, biaya akan membengkak.
Kecepatan	Sangat cepat (~0.21 ms per data, total 42 ms untuk 200 data). Tidak bergantung internet.	Lambat (~1.522 ms per data, total 5 menit untuk 200 data) karena harus mengirim request ke server cloud.
Kebutuhan Data Latih	Butuh data berlabel cukup banyak (800 baris) agar bobot model bisa belajar dengan baik.	Bisa bekerja dengan sedikit data. Cukup beri instruksi dan 6 contoh data di prompt (few-shot).
Privasi Data	Aman. Data mahasiswa tetap berada di server lokal kampus.	Ada risiko privasi karena data nilai dan presensi mahasiswa dikirim ke server pihak ketiga di cloud.
Konsistensi Output	Konsisten 100%. Model file .pkl selalu memberi hasil yang sama untuk input yang sama.	Bisa berubah jika penyedia cloud memperbarui versi model di backend.
Fleksibilitas Fitur	Kaku. Menambah data teks (misal catatan dosen wali) perlu arsitektur baru dan training ulang.	Sangat fleksibel. Catatan teks tambahan bisa langsung diselipkan ke dalam prompt.

Pembahasan Ringkas

Kedua model sama-sama paling kesulitan di kelas Waspada. Mahasiswa di kelas ini memiliki nilai yang saling bertolak belakang (misalnya IPK cukup tinggi tetapi presensi rendah, atau sebaliknya presensi rajin tetapi nilai UTS kecil). DNN memproses kondisi ini dengan bobot kompromi terbaik sehingga meraih F1 0.6981. Sedangkan LLM justru memakai Waspada sebagai jawaban aman setiap kali ragu, sehingga akurasi dan recall kelas lainnya ikut rusak.

Untuk data tabel angka terstruktur, DNN menang telak dalam hal akurasi, kecepatan (7.000 kali lebih cepat), dan biaya. Namun keunggulan LLM ada pada kemudahan awal: jika kampus belum punya dataset berlabel 800 data, LLM bisa langsung dipakai dengan beberapa contoh saja tanpa perlu proses training berhari-hari.



Gambar 1: Perbandingan Confusion Matrix DNN vs LLM Service 1 vs GPT-4o-mini

3. Penggantian AI Service dan Komparasi Tri-Model

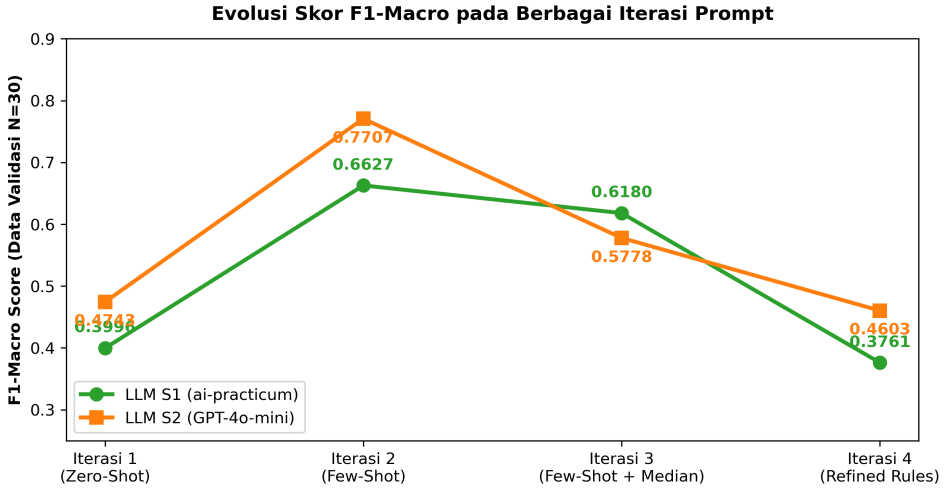
Sesuai tugas nomor 3, saya mengganti AI service di Modul 2A dari ai-practicum ke OpenAI GPT-4o-mini. Model ini dipilih karena merupakan standar industri untuk model ringkas yang cepat dan murah.

Hasil Iterasi Prompt pada Data Validasi (N=30)

Kedua AI service diuji menggunakan 4 iterasi prompt yang sama pada 30 data validasi dari data train:

Versi Prompt	Akurasi (S1)	F1 Macro (S1)	Akurasi (GPT)	F1 Macro (GPT)
Iterasi 1 (Zero-Shot)	0.4667	0.3996	0.5667	0.4743
Iterasi 2 (Few-Shot 6 Contoh)	0.6667	0.6627 (Terbaik)	0.7667	0.7707 (Terbaik)
Iterasi 3 (Few-Shot + Median)	0.6333	0.6180	0.6000	0.5778
Iterasi 4 (Aturan Manual)	0.4667	0.3761 (Turun)	0.5000	0.4603 (Turun)

Mengapa Iterasi 4 Malah Turun? Pada iterasi 4, saya menambahkan aturan batasan manual (misalnya: "Berisiko jika IPK < 2.75 ATAU kehadiran < 75% ATAU UTS < 50..."). Skor F1 kedua model justru merosot tajam (S1 turun ke 0.3761, GPT-4o-mini turun ke 0.4603). Hal ini terjadi karena aturan logika teks yang panjang dengan banyak kata "ATAU" membingungkan model bahasa saat menimbang atribut yang saling bertentangan. Memberi contoh konkret lewat few-shot (Iterasi 2) terbukti jauh lebih efektif daripada mendikte aturan logika yang kaku.



Gambar 2: Skor F1 Macro pada 4 Iterasi Prompt (Data Validasi)

Tabel Perbandingan 3 Model pada Data Test (200 Sampel)

Berikut tabel perbandingan hasil evaluasi akhir pada 200 data test yang sama setelah menambahkan OpenAI GPT-4o-mini:

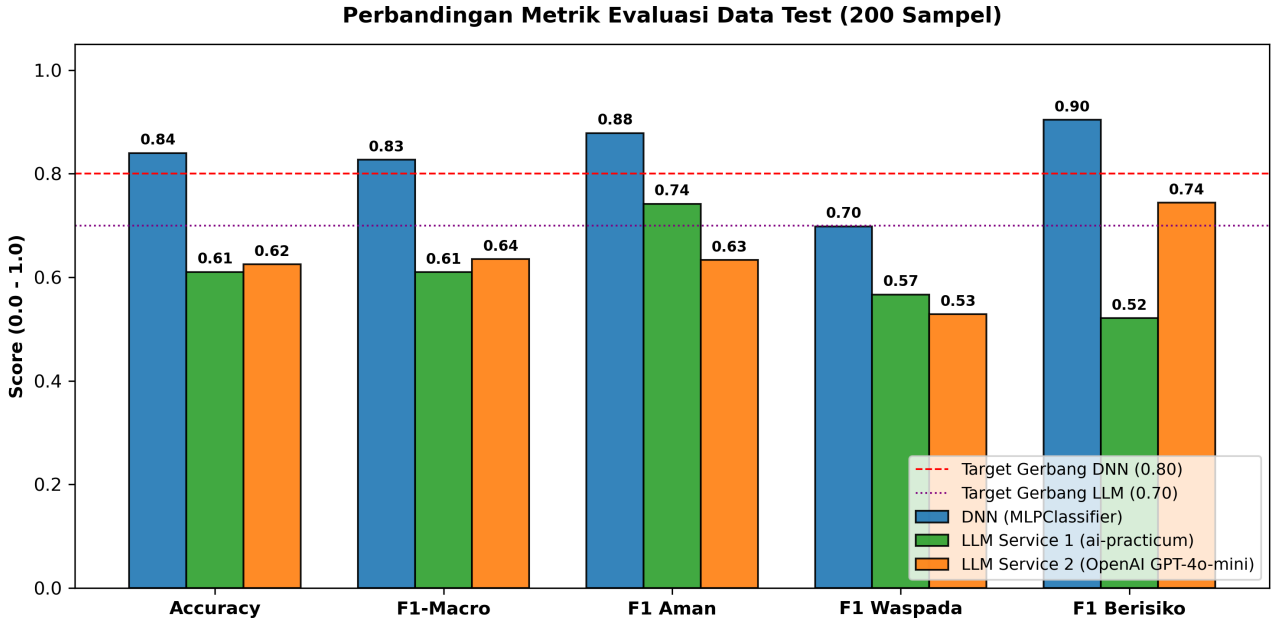
Metrik Evaluasi	DNN (Modul 1A)	LLM S1 (ai-practicum)	LLM S2 (GPT-4o-mini)
Akurasi Keseluruhan	0.8400 (84.00%)	0.6100 (61.00%)	0.6250 (62.50%)
F1-Score (Macro)	0.8269 (82.69%)	0.6098 (60.98%)	0.6351 (63.51%)
F1 Aman	0.8784	0.7419	0.6333
F1 Waspada	0.6981	0.5667	0.5283
F1 Berisiko	0.9041	0.5208	0.7438 (+22.30%)
Recall Berisiko	0.9296 (93.0%)	0.3521 (35.2%)	0.6338 (63.4%)
Jawaban Invalid	0 (0.0%)	0 (0.0%)	0 (0.0%)
Waktu 200 Data Uji	0.042 detik (42 ms)	304 detik (5 menit)	33 detik (33 detik)
Waktu Rata-rata / Data	0.21 ms	1.522 ms	165 ms (9x lebih cepat)
File Konfigurasi	model_status_akademik.pkl	config.json	config_service2.json

Analisis Penambahan AI Service 2 (GPT-4o-mini)

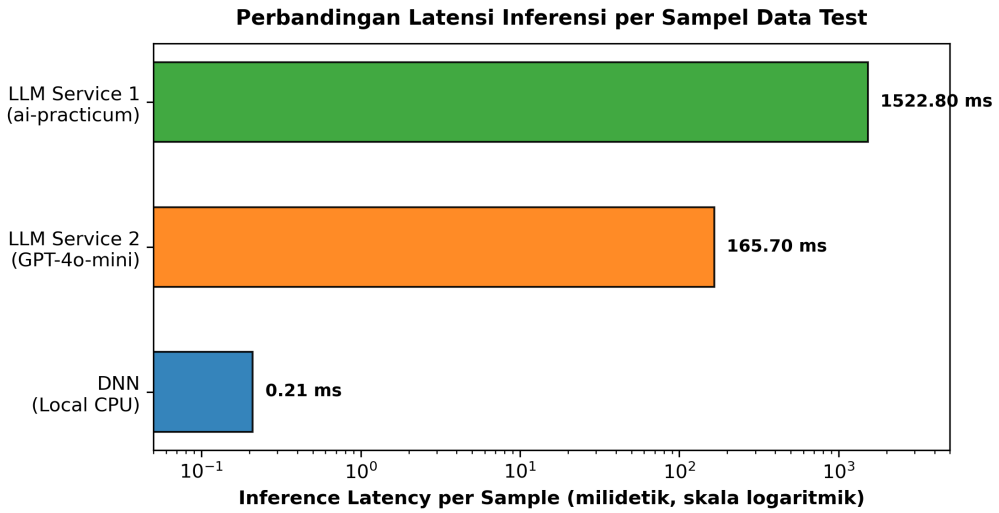
1. GPT-4o-mini Memperbaiki Deteksi Kelas Berisiko: Masalah terbesar pada Service 1 adalah rendahnya deteksi mahasiswa Berisiko (recall hanya 35.21%). Saat diganti ke GPT-4o-mini, recall kelas Berisiko naik ke 63.38%, dan F1 Berisiko naik dari 0.5208 menjadi 0.7438 (+22.30%). GPT-4o-mini lebih tegas mendeteksi mahasiswa yang bermasalah dan tidak melempar semua keraguan ke status Waspada.

2. Jauh Lebih Cepat: GPT-4o-mini menyelesaikan 200 data test hanya dalam 33 detik (165 milidetik per data), sekitar 9 kali lebih cepat dibanding ai-practicum yang membutuhkan waktu 5 menit. Kecepatan ini penting agar aplikasi web Streamlit pada Modul 2B terasa responsif.

3. DNN Tetap Paling Akurat: Meskipun GPT-4o-mini lebih baik dari Service 1, akurasinya (62.50%) tetap belum bisa menyaingi DNN (84.00%). Hasil ini membuktikan bahwa untuk data tabel angka, model machine learning khusus seperti DNN tetap menjadi pilihan utama.



Gambar 3: Perbandingan Akurasi dan F1 per Kelas pada 200 Data Test



Gambar 4: Perbandingan Kecepatan Waktu Inferensi per Data (Skala Logaritmik)

4. Kesimpulan

1. DNN Terbukti Terbaik untuk Data Tabel Angka: Model Deep Neural Network (MLPClassifier 64-32-16) meraih akurasi tertinggi (84.00%) dan F1 macro 82.69%, serta waktu inferensi instan 0.21 milidetik per data tanpa biaya API. Model ini lolos gerbang evaluasi (F1 >= 0.80) dan siap digunakan untuk deploy di Streamlit pada Modul 1B.

2. Karakteristik LLM: LLM siap pakai bisa langsung digunakan tanpa training data ratusan baris, tetapi akurasinya di data tabel masih terbatas (61-62%) dan rawan bias ke kelas tengah (Waspada). Prompt few-shot (Iterasi 2) terbukti paling efektif, sedangkan mendikte aturan panjang (Iterasi 4) justru menurunkan skor.

3. Penggantian AI Service: Penggunaan OpenAI GPT-4o-mini memperbaiki deteksi kelas Berisiko (F1 naik ke 0.7438) dan berjalan 9 kali lebih cepat dibanding service bawaan praktikum.

4. Saran Penerapan: Di sistem kampus, kombinasi terbaik adalah memakai DNN untuk skrining massal seluruh mahasiswa tiap semester karena cepat, akurat, dan gratis. Setelah itu, LLM dapat dipakai untuk membuat rekomendasi belajar atau draf pesan personal bagi mahasiswa yang terdeteksi Waspada atau Berisiko.

5. Keamanan: Seluruh pengerjaan praktikum dan dokumen laporan ini dipastikan bersih tanpa menyertakan API Key privat. Kunci disimpan lewat file .env yang diabaikan oleh git.